Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
про виконання лабораторної роботи №5
«Обробка помилок, файли та серіалізація»
з дисципліни «Професійний Python»

Виконав:
студент 3 курсу
групи ФЕП-33
Зінчук Остап

Викладач:
проф. Свелеба Сергій Андрійович

Львів – 2026

2. Тема лабораторної роботи

Обробка помилок, файли та серіалізація.

3. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних (CSV, JSON, YAML). Опанувати механізм виключень, власну ієрархію custom exceptions, exception chaining, принципи EAFP і fail-fast, контекстні менеджери, модуль logging з маскуванням чутливих банківських даних, streaming I/O, атомарний запис файлів та валідацію вхідних даних на межі системи.

4. Номер індивідуального варіанта

Варіант 4: Імпорт та експорт системи банківських рахунків.

5. Постановка задачі

Розробити надійний модуль імпорту, валідації, фільтрації та експорту банківських транзакцій. Модуль повинен: читати дані з CSV-файлу у потоковому режимі; валідувати номер рахунку, клієнта, початковий баланс (>= 0), дозволені операції (deposit/withdraw) та суму операції (> 0); гарантувати недопущення від'ємного залишку після операції списання (withdraw); підтримувати зовнішню YAML-конфігурацію; підтримувати політики Strict та Tolerant mode; не завантажувати весь файл у пам'ять; забезпечувати атомарний експорт у JSON (зберігаючи попередній файл неушкодженим при збої); вести журнал подій через logging із маскуванням конфіденційних реквізитів рахунків; перевіряти якість коду через flake8 і mypy.

6. Input data schema (Схема вхідних даних)

Структура полів вхідного файлу CSV:
• account_number: рядковий ідентифікатор банківського рахунку (обов'язковий, не порожній);
• client: прізвище та ім'я власника рахунку (обов'язкове, не порожнє);
• balance: початковий баланс рахунку (дійсне число >= 0);
• operation: тип операції (дозволені значення: deposit, withdraw);
• amount: сума транзакції (дійсне число > 0).
Правило розрахунку: при deposit новий баланс = balance + amount; при withdraw якщо balance < amount, фіксується помилка овердрафту (заборона від'ємного балансу), інакше новий баланс = balance - amount.

7. Приклад CSV файлу (data/accounts.csv)

```
account_number,client,balance,operation,amount
UA0000001,Іван Петренко,1500.50,deposit,200.00
UA0000002,Олена Коваль,500.00,withdraw,100.00
UA0000003,Василь Бойко,300.00,withdraw,600.00
UA0000004,Анна Мельник,2500.00,deposit,500.00
INVALID_1,,500.00,deposit,100.00
UA0000005,Тарас Шевченко,invalid_balance,deposit,100.00
UA0000006,Леся Українка,-50.00,deposit,100.00
UA0000007,Іван Франко,1200.00,unknown_operation,100.00
UA0000008,Михайло Грушевський,3400.00,deposit,-50.00
UA0000009,Богдан Хмельницький,850.00,withdraw,150.00
UA0000010,Соломія Крушельницька,12000.00,deposit,3000.00
```

8. Структура проєкту

```
Lab_05/
├── .gitignore
├── pyproject.toml
├── config.yaml
├── README.md
├── data/
│   └── accounts.csv
├── output/
│   └── accounts.json
├── logs/
│   └── application.log
├── report/
│   └── Проф. Python-Lab_05-Зінчук.pdf
├── src/
│   └── reliable_importer/
│       ├── __init__.py
│       ├── config.py
│       ├── exceptions.py
│       ├── exporters.py
│       ├── logging_config.py
│       ├── main.py
│       ├── models.py
│       ├── readers.py
│       ├── services.py
│       ├── utils.py
```

```
│       └── validators.py
└── tests/
    └── test_importer.py
```

9-10. Exception hierarchy та пояснення custom exceptions

Для системи створено доменну ієрархію виключень, успадковану від ApplicationError. Клас RecordValidationError містить контекстні атрибути line_number та field, що дозволяє точно діагностувати помилки у вхідному файлі.

```
"""Модуль доменної ієрархії виключень (Custom Exception Hierarchy)."""

class ApplicationError(Exception):
    """Базове виключення додатку."""

class ConfigurationError(ApplicationError):
    """Помилка валідації або завантаження конфігурації."""

class DataError(ApplicationError):
    """Базове виключення обробки даних."""

class DataImportError(DataError):
    """Помилка читання або імпорту вхідних даних."""

class RecordValidationError(DataError):
    """Помилка валідації окремого запису з контекстом номеру рядка та поля."""

    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

class DataExportError(DataError):
    """Помилка експорту або збереження вихідних даних."""
```

11. Приклад Exception Chaining (збереження першопричини)

Використання конструкції raise ... from error прив'язує вихідне виключення (наприклад, ValueError) до доменного RecordValidationError через атрибут __cause__:

```
bal_raw = row.get('balance', '')
try:
    balance = float(bal_raw)
except (TypeError, ValueError) as error:
    raise RecordValidationError(
        f'Баланс повинен бути числом: {bal_raw!r}',
        line_number=line_number,
        field='balance',
    ) from error
```

12-14. Код configuration loader, вміст config.yaml та валідація

Вміст config.yaml:

```
input:
  path: data/accounts.csv

output:
  path: output/accounts.json

processing:
  minimum_balance: 100.0
  skip_invalid: true

logging:
  level: INFO
  path: logs/application.log
```

Модуль завантаження та Fail-Fast перевірки (config.py):

```
"""Модуль завантаження та валідації YAML конфігурації."""

from pathlib import Path
import yaml

from .exceptions import ConfigurationError
from .models import AppConfig, LoggingConfig, ProcessingConfig

VALID_LOG_LEVELS = {"DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"}

def validate_config(config: AppConfig) -> None:
    """Перевіряє коректність параметрів конфігурації (Fail-Fast)."""
```

```
    if config.processing.minimum_balance < 0:
        raise ConfigurationError(
            "Параметр minimum_balance не може бути від'ємним"
        )

    if not config.input_path.exists():
        raise ConfigurationError(
            f"Вхідний файл не знайдено: {config.input_path}"
        )

    if config.logging.level.upper() not in VALID_LOG_LEVELS:
        raise ConfigurationError(
            f"Невідомий рівень логування: {config.logging.level}"
        )

def load_config(path: Path) -> AppConfig:
    """Завантажує та валідує конфігурацію з YAML-файлу."""
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(
            f"Файл конфігурації не знайдено: {path}"
        ) from error
    except yaml.YAMLError as error:
        raise ConfigurationError(
            "Некоректний синтаксис YAML конфігурації"
        ) from error

    if not isinstance(raw, dict):
        raise ConfigurationError(
            "Конфігураційний файл повинен містити словник"
        )

    try:
        input_raw = raw["input"]["path"]
        output_raw = raw["output"]["path"]
        proc_raw = raw["processing"]
        log_raw = raw["logging"]

        config = AppConfig(
            input_path=Path(input_raw),
            output_path=Path(output_raw),
            processing=ProcessingConfig(
                minimum_balance=float(proc_raw["minimum_balance"]),
                skip_invalid=bool(proc_raw["skip_invalid"]),
            ),
            logging=LoggingConfig(
                level=str(log_raw["level"]),
                path=Path(log_raw["path"]),
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError(
            "Структура конфігураційного файлу є невалідною"
        ) from error

    validate_config(config)
    return config
```

15-16. Logging configuration, захист конфіденційних реквізитів та приклад логу

Модуль logging_config.py налаштовує вивід у файл та консоль, а функція mask_sensitive_account() маскує номери рахунків (UA00****0003) для дотримання банківської таємниці та вимог варіанта 4:

```
"""Модуль налаштування журналювання із захистом конфіденційних даних."""

import logging
from pathlib import Path

def mask_sensitive_account(account_number: str) -> str:
    """Маскує номер банківського рахунку (наприклад UA12****7890)."""
    clean = account_number.strip()
    if len(clean) <= 6:
        return "***"
    prefix = clean[:4]
    suffix = clean[-4:]
    return f"{prefix}****{suffix}"

def configure_logging(level: str, path: Path) -> None:
    """Конфігурує журналювання у файл та консоль з форматуванням."""
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)

    file_formatter = logging.Formatter(
        "%(asctime)s | %(levelname)s | %(name)s | %(message)s"
    )
```

```
    console_formatter = logging.Formatter(
        "%(asctime)s | %(levelname)s | %(message)s"
    )

    file_handler = logging.FileHandler(path, encoding="utf-8")
    file_handler.setLevel(numeric_level)
    file_handler.setFormatter(file_formatter)

    console_handler = logging.StreamHandler()
    console_handler.setLevel(numeric_level)
    console_handler.setFormatter(console_formatter)

    root_logger = logging.getLogger()
    root_logger.setLevel(numeric_level)
    root_logger.handlers.clear()
    root_logger.addHandler(file_handler)
    root_logger.addHandler(console_handler)
```

Приклад записів у logs/application.log:

```
2026-11-01 15:31:59,518 | INFO | reliable_importer.utils | Операцію 'Імпорт та експорт банківських рахунків' розпочато
2026-11-01 15:31:59,519 | WARNING | reliable_importer.services | Невалідний запис у рядку 4 (рахунок UA00****0003, поле amount): Недост
2026-11-01 15:31:59,519 | WARNING | reliable_importer.services | Невалідний запис у рядку 6 (рахунок INVA****ID_1, поле client): Ім'я к
2026-11-01 15:31:59,519 | WARNING | reliable_importer.services | Невалідний запис у рядку 7 (рахунок UA00****0005, поле balance): Балан
2026-11-01 15:31:59,552 | WARNING | reliable_importer.services | Невалідний запис у рядку 8 (рахунок UA00****0006, поле balance): Балан
2026-11-01 15:31:59,552 | WARNING | reliable_importer.services | Невалідний запис у рядку 9 (рахунок UA00****0007, поле operation): Нев
2026-11-01 15:31:59,552 | WARNING | reliable_importer.services | Невалідний запис у рядку 10 (рахунок UA00****0008, поле amount): Сума
2026-11-01 15:31:59,553 | INFO | reliable_importer.utils | Операцію 'Імпорт та експорт банківських рахунків' успішно завершено за 0.034
2026-11-01 15:31:59,553 | INFO | __main__ | Підсумок обробки: Всього=11, Валідних=5, Невалідних=6, Експортовано=5
```

17-18. Streaming reader та код валідації (readers.py, validators.py)

```
"""Модуль потокового зчитування зовнішніх CSV та JSONL файлів."""

from collections.abc import Iterator
import csv
import json
from pathlib import Path

from .exceptions import DataImportError

def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    """Потоково повертає кортеж (номер рядка, словник) із CSV-файлу."""
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, row
    except OSError as error:
        raise DataImportError(
            f"Помилка читання вхідного файлу: {path}"
        ) from error

def read_jsonl_rows(path: Path) -> Iterator[tuple[int, dict[str, object]]]:
    """Потоково повертає (номер рядка, об'єкт) із JSON Lines файлу."""
    try:
        with path.open("r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                clean_line = line.strip()
                if not clean_line:
                    continue
                try:
                    data = json.loads(clean_line)
                    if isinstance(data, dict):
                        yield line_number, data
                except json.JSONDecodeError as error:
                    raise DataImportError(
                        f"Некоректний JSON рядок
                    ) from error
    except OSError as error:
        raise DataImportError(
            f"Помилка читання JSONL файлу: {path}"
        ) from error
```

```
"""Модуль валідації записів згідно бізнес-правил предметної області."""

from .exceptions import RecordValidationError
from .models import BankAccountRecord

ALLOWED_OPERATIONS = {"deposit", "withdraw"}

def parse_account(
    row: dict[str, str],
    *,
    line_number: int,
) -> BankAccountRecord:
    """Валідує поля банківського запису згідно правил Варіанта 4:

    - Номер рахунку та клієнт не порожні
    - Початковий баланс balance >= 0
    - Дозволені операції: deposit, withdraw
    - Сума операції amount > 0
    - Заборона від'ємного балансу після операції withdraw
    """
    account_number = row.get("account_number", "").strip()
    if not account_number:
        raise RecordValidationError(
            "Номер рахунку не може бути порожнім",
            line_number=line_number,
            field="account_number",
        )

    client = row.get("client", "").strip()
    if not client:
        raise RecordValidationError(
            "Ім'я клієнта не може бути порожнім",
            line_number=line_number,
            field="client",
        )

    bal_raw = row.get("balance", "")
    try:
        balance = float(bal_raw)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(
            f"Баланс повинен бути числом: {bal_raw!r}",
            line_number=line_number,
            field="balance",
```

```
        ) from error

    if balance < 0:
        raise RecordValidationError(
            f"Баланс не може бути від'ємним: {balance}",
            line_number=line_number,
            field="balance",
        )

    operation = row.get("operation", "").strip().lower()
    if operation not in ALLOWED_OPERATIONS:
        msg = (
            f"Невідома операція {operation!r}. "
            f"Дозволені: {ALLOWED_OPERATIONS}"
        )
        raise RecordValidationError(
            msg,
            line_number=line_number,
            field="operation",
        )

    amt_raw = row.get("amount", "")
    try:
        amount = float(amt_raw)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(
            f"Сума транзакції повинна бути числом: {amt_raw!r}",
            line_number=line_number,
            field="amount",
        ) from error

    if amount <= 0:
        raise RecordValidationError(
            f"Сума транзакції повинна бути більшою за нуль: {amount}",
            line_number=line_number,
            field="amount",
        )

    if operation == "deposit":
        new_balance = round(balance + amount, 2)
    elif operation == "withdraw":
        if balance < amount:
            msg = (
                f"Недостатньо коштів: баланс {balance:.2f} < "
                f"суми зняття {amount:.2f} (заборона від'ємного балансу)"
            )
            raise RecordValidationError(
                msg,
                line_number=line_number,
                field="amount",
            )
        new_balance = round(balance - amount, 2)
    else:
        new_balance = balance

    return BankAccountRecord(
        account_number=account_number,
        client=client,
        balance=balance,
        operation=operation,
        amount=amount,
        new_balance=new_balance,
    )
```

19-21. Strict та Tolerant processing, статистика обробки (services.py)

```
"""Модуль сервісів імпорту, обробки та фільтрації даних."""

from collections.abc import Iterable, Iterator
from dataclasses import dataclass
import logging
from pathlib import Path

from .exceptions import RecordValidationError
from .logging_config import mask_sensitive_account
from .models import BankAccountRecord
from .readers import read_csv_rows
from .validators import parse_account

logger = logging.getLogger(__name__)

@dataclass
class ImportStatistics:
    """Лічильник статистики обробки записів."""

    total: int = 0
    valid: int = 0
```

```
    invalid: int = 0
    exported: int = 0

def import_accounts(
    path: Path,
    *,
    skip_invalid: bool,
    statistics: ImportStatistics,
) -> Iterator[BankAccountRecord]:
    """Потоковий імпорт банківських записів з підтримкою Strict / Tolerant:

    - Strict mode (skip_invalid=False): перша помилка перериває процес.
    - Tolerant mode (skip_invalid=True): некоректні записи логуються та
      пропускаються.
    """
    for line_number, row in read_csv_rows(path):
        statistics.total += 1
        try:
            account = parse_account(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
            acc_raw = row.get("account_number", "")
            masked = mask_sensitive_account(acc_raw)
            logger.warning(
                "Невалідний запис у рядку %s (рахунок %s, поле %s): %s",
                error.line_number,
                masked,
                error.field,
                error,
            )
            if skip_invalid:
                continue
            raise

        statistics.valid += 1
        yield account

def filter_by_balance(
    accounts: Iterable[BankAccountRecord],
    minimum_balance: float,
) -> Iterator[BankAccountRecord]:
    """Потокова фільтрація рахунків за мінімальним балансом."""
    for account in accounts:
        if account.balance >= minimum_balance:
            yield account
```

22-23. JSON serialization та Atomic Export (exporters.py)

```
"""Модуль атомарного експорту у JSON та звітів про помилки."""

from collections.abc import Iterable
import json
from pathlib import Path

from .exceptions import DataExportError
from .models import BankAccountRecord

def export_accounts_json(
    accounts: Iterable[BankAccountRecord],
    path: Path,
) -> int:
    """Атомарно записує потік рахунків у вихідний JSON масив:

    - Створює тимчасовий файл .tmp
    - Здійснює потоковий запис масиву
    - Атомарно замінює цільовий файл
    - У разі помилки видаляє тимчасовий файл та зберігає цільовий цілим
    """
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0

    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for account in accounts:
                if not first:
                    file.write(",\n")
                payload_str = json.dumps(
                    account.to_dict(),
                    ensure_ascii=False,
                    indent=2,
                )
                file.write(payload_str)
                first = False
                count += 1
```

```
            file.write("\n]\n")

        temporary.replace(path)
    except Exception as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(
            f"Не вдалося виконати атомарний експорт у {path}"
        ) from error

    return count
```

24. Власний Context Manager (utils.py)

```
"""Модуль утиліт та власних context managers."""

from collections.abc import Iterator
from contextlib import contextmanager
import logging
from time import perf_counter

logger = logging.getLogger(__name__)

@contextmanager
def logged_operation(operation: str) -> Iterator[None]:
    """Власний контекстний менеджер для фіксації початку, завершення,

    тривалості або збою операції з реєстрацією у логах.
    """
    logger.info("Операцію '%s' розпочато", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("Операція '%s' завершилася збоєм", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info(
            "Операцію '%s' успішно завершено за %.6f с",
            operation,
            elapsed,
        )
```

25-26. Консольний вивід виконання програми (main.py)

```
2026-11-01 15:31:59,518 | INFO | Операцію 'Імпорт та експорт банківських рахунків' розпочато
2026-11-01 15:31:59,519 | WARNING | Невалідний запис у рядку 4 (рахунок UA00****0003, поле amount): Недостатньо коштів...
2026-11-01 15:31:59,519 | WARNING | Невалідний запис у рядку 6 (рахунок INVA****ID_1, поле client): Ім'я клієнта не може бути порожнім
2026-11-01 15:31:59,519 | WARNING | Невалідний запис у рядку 7 (рахунок UA00****0005, поле balance): Баланс повинен бути числом
2026-11-01 15:31:59,552 | WARNING | Невалідний запис у рядку 8 (рахунок UA00****0006, поле balance): Баланс не може бути від'ємним
2026-11-01 15:31:59,552 | WARNING | Невалідний запис у рядку 9 (рахунок UA00****0007, поле operation): Невідома операція
2026-11-01 15:31:59,552 | WARNING | Невалідний запис у рядку 10 (рахунок UA00****0008, поле amount): Сума транзакції повинна бути більш
2026-11-01 15:31:59,553 | INFO | Операцію 'Імпорт та експорт банківських рахунків' успішно завершено за 0.034467 с
2026-11-01 15:31:59,553 | INFO | Підсумок обробки: Всього=11, Валідних=5, Невалідних=6, Експортовано=5
============================================================
Обробку даних завершено успішно.
Всього: 11 | Валідних: 5 | Невалідних: 6 | Експортовано: 5
Результат збережено у: output\accounts.json
============================================================
```

27. Результати Experiment 1: Strict vs Tolerant Mode

Параметр	Strict Mode (skip_invalid: false)	Tolerant Mode (skip_invalid: true)
[redacted]	1001	1010
[redacted] (Valid)	1000	1000
[redacted] (Invalid)	1	10
[redacted] (Exported)	0	1000
[redacted]	[redacted] (SystemExit/Exception)	[redacted]

28-29. Результати Experiment 2: Streaming vs Eager (таблиця пам'яті)

Кількість записів	Eager Time (s)	Streaming Time (s)	Eager Peak RAM	Streaming Peak RAM
10 000	0.0693	0.0655	4.64 MB	0.19 MB
100 000	0.6442	0.6040	44.96 MB	0.19 MB
500 000	5.1786	5.8735	224.34 MB	0.19 MB

30. Відповіді на контрольні запитання (розділ 90 методички)

1. Що таке exception?

Об'єкт у Python, який сигналізує про виникнення ненормальної ситуації або помилки під час виконання програми.

2. Для чого використовується try?

Для позначення блоку коду, в якому можуть виникнути потенційні виключні ситуації, які потрібно перехопити.

3. Для чого except?

Для визначення блоку обробки конкретного типу виключення, що виникло в блоці try.

4. Коли виконується else після try?

Блок else виконується тільки тоді, коли блок try завершився успішно без виникнення жодного виключення.

5. Коли виконується finally?

Блок finally виконується обов'язково завжди, незалежно від того, виникло виключення, було воно оброблене чи ні.

6. Для чого використовується raise?

Для явного підняття (виклику) виключення в коді програми.

7. Що означає raise без аргументів?

Повторне підняття поточного активного виключення в блоці except для передачі його вгору за стеком викликів.

8. Чому не рекомендується except: pass?

Приховує помилки програмування (наприклад, помилки синтаксису, NameError, SystemExit), робить налагодження вкрай важким.

9. Чому потрібно перехоплювати конкретні exception types?

Щоб коректно обробляти тільки очікувані ненормальні ситуації, не маскуючи непередбачені збої.

10. Що таке exception hierarchy?

Ієрархія класів виключень, побудована через наслідування від Exception, що структурує помилки за типами.

11. Для чого створювати custom exceptions?

Щоб надати помилкам зрозумілий доменний зміст предметної області та дозволити верхнім рівням системи селективно їх обробляти.

12. Що таке exception chaining?

Механізм зв'язування виключення високого рівня з первинною причиною за допомогою конструкції raise ... from cause.

13. Що означає raise ... from ...?

Явно встановлює атрибут __cause__ для нового виключення, зберігаючи повну історію першопричини в traceback.

14. Яку перевагу дає chaining для debugging?

Дозволяє розробнику одночасно бачити як зрозуміле доменне повідомлення, так і технічну першопричину з точним місцем збою.

15. Що означає EAFP?

Easier to Ask for Forgiveness than Permission: підхід Python, за якого спочатку виконують дію в блоці try, а винятки перехоплюють.

16. Що означає LBYL?

Look Before You Leap: підхід з попередньою перевіркою всіх умов (if) перед виконанням операції.

17. Що таке fail-fast?

Принцип проектування, за якого програма негайно зупиняє роботу при виявленні некоректного стану на ранньому етапі.

18. Де доцільно застосовувати fail-fast?

При завантаженні конфігурації, перевірці існування критичних файлів, підключенні до бази даних на старті додатку.

19. Що таке context manager?

Об'єкт, що керує життєвим циклом ресурсу, забезпечуючи гарантоване виконання дій до і після використання ресурсу.

20. Для чого використовується with?

Для безпечного керування ресурсами через контекстні менеджери з автоматичним звільненням ресурсів.

21. Які methods утворюють context manager protocol?

Методи __enter__() та __exit__().

22. Що робить __enter__()?

Ініціалізує контекст, виділяє ресурс і повертає об'єкт для прив'язки через ключове слово as.

23. Що робить __exit__()?

Звільняє або закриває ресурс, приймає інформацію про виключення і вирішує, чи пригнічувати його (повертаючи True/False).

24. Чи закриває with файл при exception?

Так, метод __exit__() викликається гарантовано навіть при аварійному виході через виключення.

25. Для чого contextlib.contextmanager?

Декоратор для створення простих контекстних менеджерів за допомогою генераторних функцій з yield.

26. Яку роль має yield у generator-based context manager?
Розділяє фазу ініціалізації (__enter__) та фазу очищення (__exit__), передаючи керування тілу with.

27. Що таке pathlib.Path?
Об'єктно-орієнтоване представлення файлового шляху у стандартній бібліотеці Python.

28. Чому pathlib кращий за ручну конкатенацію шляхів?
Забезпечує кросплатформеність (роздільники /, \), надає безпечні методи навігації та маніпуляції файловою системою.

29. Для чого задається encoding='utf-8'?
Щоб уникнути залежності від системного кодування ОС за замовчуванням (наприклад, cp1251 або ASCII) та коректно обробляти кирилицю.

30. Що таке CSV?
Comma-Separated Values — текстовий формат представлення табличних даних, де колонки розділяються комами або іншими символами.

31. Для чого csv.reader?
Для читання CSV-файлів у вигляді списків рядкових значень.

32. Для чого csv.DictReader?
Для читання CSV-файлів у вигляді словників, де ключами виступають назви колонок із заголовка.

33. Чим DictReader зручний для structured data?
Дозволяє звертатися до полів за іменами замість числових індексів, що підвищує читабельність і стійкість до зміни порядку колонок.

34. Для чого csv.DictWriter?
Для запису структурованих словників у CSV-файл із автоматичним узгодженням заголовків.

35. Що таке JSON?
JavaScript Object Notation — легкий текстовий формат обміну структурованими даними.

36. Що таке serialization?
Процес перетворення об'єкта програми або структури даних у байти чи текст для передачі або збереження.

37. Що таке deserialization?
Процес відновлення повноцінного об'єкта програми або структури даних із збереженого формату.

38. Чим json.dump() відрізняється від json.dumps()?
json.dump записує дані безпосередньо у відкритий файловий потік, а json.dumps серіалізує об'єкт у пам'ять у вигляді рядка str.

39. Чим json.load() відрізняється від json.loads()?
json.load зчитує та десеріалізує дані з відкритого файлового дескриптора, а json.loads приймає готовий рядок str.

40. Що таке JSONDecodeError?
Виключення, що піднімається модулем json при спробі розпарсити некоректно сформований JSON-текст.

41. Для чого використовується YAML?
Зручний для читання людиною формат розмітки даних, який широко застосовується для конфігураційних файлів.

42. Чому рекомендується yaml.safe_load()?
Запобігає виконанню довільного коду Python, безпечно парсячи тільки стандартні базові типи даних.

43. Які типові сценарії використання JSON?
REST API, передача даних між мікросервісами, збереження структурованих результатів, веб-інтерфейси.

44. Які типові сценарії використання YAML?
Конфігураційні файли додатків (config.yaml), Docker Compose, Kubernetes manifests, CI/CD конвеєри.

45. Що таке configuration file?
Зовнішній файл із параметрами програми, який дозволяє змінювати її поведінку без модифікації коду.

46. Чому configuration краще не hard-code у Python?
Зміна параметрів не вимагає перекомпіляції чи перепакування коду, спрощує підтримку різних середовищ (dev/stage/prod).

47. Що потрібно перевіряти у configuration?
Наявність обов'язкових ключів, типи значень, допустимі межі чисел, існування вказаних шляхів до файлів.

48. Що таке logging?
Процес фіксації подій, інформаційних повідомлень, попереджень та помилок під час роботи програми.

49. Чим logging відрізняється від print()?
Підтримує рівні важливості, налаштування форматів, маршрутизацію у файли чи мережу, фільтрацію та ротацію.

50. Які існують logging levels?
DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?
Для фіксації штатних подій нормального виконання програми (початок операції, завершення, підсумки).

52. Для чого logger.warning()?
Для фіксації небажаних, але не критичних ситуацій (пропущений невалідний рядок, дефолтне значення).

53. Для чого logger.error()?
Для реєстрації помилок, які порушили виконання окремої операції, але не зупинили весь додаток.

54. Для чого logger.exception()?
Викликається всередині блоку except; автоматично додає до повідомлення повний traceback виключення.

55. Що містить traceback?
Послідовність викликів функцій від точки виникнення помилки до поточного місця зі шляхами, номерами рядків та кодом.

56. Чому secrets не можна записувати в logs?
Щоб запобігти витоку паролів, токенів, API-ключів та персональних банківських даних у відкритий журнал подій.

57. Що таке streaming I/O?
Потокова обробка даних частинами (по рядках/блоках) без завантаження всього масиву у пам'ять одночасно.

58. Яка перевага streaming для великих files?
Константне споживання оперативної пам'яті O(1), що запобігає збою Out Of Memory (OOM).

59. Що таке eager file loading?
Жадібне завантаження, коли весь вміст файлу негайно матеріалізується у пам'ять у вигляді списку чи рядка.

60. Чому list(reader) може бути небажаним?
Матеріалізує всі рядки у список у пам'яті, що для гігабайтних файлів миттєво переповнює доступну RAM.

61. Що таке strict validation mode?
Режим, за якого перша ж виявлена помилка у вхідних даних негайно зупиняє роботу програми.

62. Що таке tolerant mode?
Режим, за якого некоректні записи фіксуються у логах, а програма продовжує обробку решти валідних даних.

63. Коли доцільно використовувати strict mode?
Для критичних фінансових транзакцій, банківських проводок та імпорту баз даних, де помилка неприпустима.

64. Коли tolerant mode?
Для обробки логів, аналітики великих масивів, телеметрії, де важливіше зібрати максимум даних.

65. Що таке atomic write?
Техніка запису спочатку у тимчасовий файл із подальшою миттєвою заміною цільового файлу.

66. Для чого використовується temporary file?
Щоб у разі аварійного збою програми не залишити частково записаний або пошкоджений вихідний файл.

67. Що може статися при звичайному output write та аварійному завершенні?
Вихідний файл виявиться обірваним або пошкодженим, що призведе до втрати коректних попередніх даних.

68. Навіщо видаляти temporary file після error?
Щоб не засмічувати дисковий простір тимчасовим сміттям та гарантувати чистоту файлової системи.

69. Що таке JSON Lines?
Текстовий формат (JSONL), де кожен рядок файлу є окремим валідним JSON-об'єктом.

70. Чому JSON Lines зручний для streaming?
Дозволяє зчитувати, валідувати та записувати дані рядок за рядком без необхідності тримати квадратні дужки масиву у пам'яті.

31. Висновок

У ході виконання лабораторної роботи №5 було успішно опановано інструменти та патерни створення надійних, відмовостійких програм мовою Python. На прикладі банківської системи (Варіант 4) розроблено модуль імпорту та експорту, побудований на принципах fail-fast та EAFP, із власною ієрархією виключень (ApplicationError, ConfigurationError, DataError, RecordValidationError, DataExportError), збереженням першопричини через exception chaining та захистом банківських реквізитів у системі логування. Забезпечено підтримку двох взаємодоповнюючих політик надійності: Strict mode (для критичних транзакцій) та Tolerant mode (для аналітичної обробки). Практичні експерименти довели фундаментальні переваги потокового введення-виведення (константні ~0.19 МБ пам'яті на датасетах у 500 000 рядків проти 224 МБ у жадібному) та підтвердили надійність атомарного експорту у запобіганні пошкодженню файлів при аварійних збоях.